



Cahier des charges technique et fonctionnel

IAM-Local — Plateforme IAM « IAM ACSI »
Gestion des identités et des accès, SSO & MFA

Version	1.0
Date	21 septembre 2026
Statut	Validé — implémentation livrée (PHP natif + MySQL)
Documentation incluse	Diagramme de classes (17 entités) · Diagrammes d'activité (SSO, Connexion & MFA)

Sommaire

1. Contexte et objectifs
2. Périmètre fonctionnel
3. Diagramme de classes (modèle de données — 17 entités)
4. Diagrammes d'activité (SSO · Connexion & MFA)
5. Architecture technique et sécurité
6. Environnement et exigences
7. Livrables et jalons

1. Contexte et objectifs

La structure met à disposition plusieurs applications Web internes (gestion de présence, gestion de salaire, gestion scolaire, etc.), chacune gérée historiquement par son propre mécanisme de connexion et d'utilisateurs. Ce cloisonnement impose une multiplicité de comptes, de mots de passe et de table d'utilisateurs à maintenir.

Le projet **IAM-Local (IAM ACSI)** vise à centraliser :

- **l'Identité** — un référentiel unique d'utilisateurs, avec organisation hiérarchique et statut de compte ;
- **la Politique de sécurité** — politique de mot de passe, verrouillage anti force brute, expiration, sessions ;
- **la Vérification renforcée (MFA)** — code OTP à usage unique envoyé par e-mail (SMTP), affiché en mode démonstration ;
- **l'Accès (SSO)** — Single Sign-On par jeton à usage unique : une connexion IAM donne accès aux applications enregistrées (accès fluide via le portail, ressaisie facultative sur lien direct) ;
- **la Gouvernance (RBAC)** — rôles, permissions, affectations utilisateur/rôle/application, journal d'audit complet ;
- **l'Administration graphique** — une console de gestion ne nécessitant aucune manipulation SQL en exploitation.

Périmètre. Le présent document décrit l'existant implémenté (version 1.0.0), qui sert de socle au déploiement et aux évolutions ultérieures.

2. Périmètre fonctionnel

2.1 Authentification

- Connexion par identifiant (e-mail) et mot de passe haché (bcrypt via `password_hash`).
- Comptes avec statuts *active* / *inactive* / *locked* / *pending*, premier mot de passe imposé (`must_change_password = 1`), expiration de mot de passe configurable.
- Anti force brute : **5 échecs autorisés** → verrouillage **30 min** (tentatives journalisées dans `login_attempts`).
- Double vérification MFA par e-mail : OTP de **6 chiffres, valable 5 minutes, 5 essais max** (`MFA_FORCE_ALL = true` , sauf administrateurs).

- **Connexion sans mot de passe** : e-mail ou téléphone + code OTP à usage unique envoyé par e-mail (`otp_login.php` , canal d'envoi `email`), verrouillage renforcé, reprise en `?resend=1` .
- Session PHP dédiée `iam_session` (durée 3600 s, inactivité 1800 s), sessions révocables depuis l'administration.

2.2 SSO applicatif

- Entrepôt `sso/authorize.php?app=CODE&return=URL[&prompt=login]` : vérifie l'application, la session IAM, le MFA, l'expiration du mot de passe et l'accès de l'utilisateur, puis émet un **jeton à usage unique** (60 s, enregistré dans `sso_tokens`).
- Fichier garde généré `iam-sso-<code>.php` (déployé automatiquement via l'administrateur ou téléchargeable ; chargé en `auto_prepend_file` via `.htaccess`) : session isolée côté application, validation du jeton via `api/sso/validate.php` , reconstruction de l'utilisateur (rôles, permissions).
- Paramètre par application `sso_prompt` (administré graphiquement) : ajoute `prompt=login` sur accès direct ⇒ ressaisie des identifiants IAM ; via le portail l'accès reste fluide.
- Vérification de la légitimité de l'URL de retour (même hôte que l'application) avant émission du jeton.

2.3 Portail & applications

- Portail (page d'accueil) : grille d'applications autorisées avec icône, couleur et description ; accès en un clic (lien SSO).
- Catalogage des applications : *code, nom, URL, redirect_uri, client_id/client_secret, active, sso_prompt, is_system*.

2.4 Gouvernance (RBAC) et administration graphique

- **Rôles & permissions** : création/édition/suppression de rôles (rôles système protégés), attribution de permissions par application.
- **Utilisateurs** : création, modification (statut, organisation, **direction / service / fonction**), réinitialisation de mot de passe, suppression, **import CSV** (modèle téléchargeable, doublons ignorés, mots de passe aléatoires, rapprochement des directions insensible aux accents).
- **Groupe**s : annuaires nommés (création, renommage, suppression, affectation des membres) ; multi-sélection dans les formulaires utilisateur.
- **Organisations** : arborescence Ministère/Direction/Service/Bureau avec affectation des utilisateurs.
- **Sessions** : consultation, filtrage par utilisateur, révocation individuelle ou totale.
- **Journal d'audit** : recherche textuelle et filtrage par action sur `audit_logs` .
- **Paramètres** : politique de mots de passe, durée de session et d'inactivité, **paramètres API & sécurité** (origines CORS autorisées, géolocalisation optionnelle des IP, anti force brute par IP).
- **E-mail & MFA** : configuration SMTP (Gmail/Outlook, clé applicative), envoi de test, guide pas à pas à 3 étapes.
- **Applications** : création, édition, permissions, accès utilisateurs, déploiement/retrait du garde SSO, téléchargement du garde.
- **OAuth / OIDC** : console de gestion des clients (identifiants, secret affiché une seule fois, URI de redirection, grants autorisés, scopes) et **rotation de la clé de signature RSA** (JWKS).

2.5 API et intégration

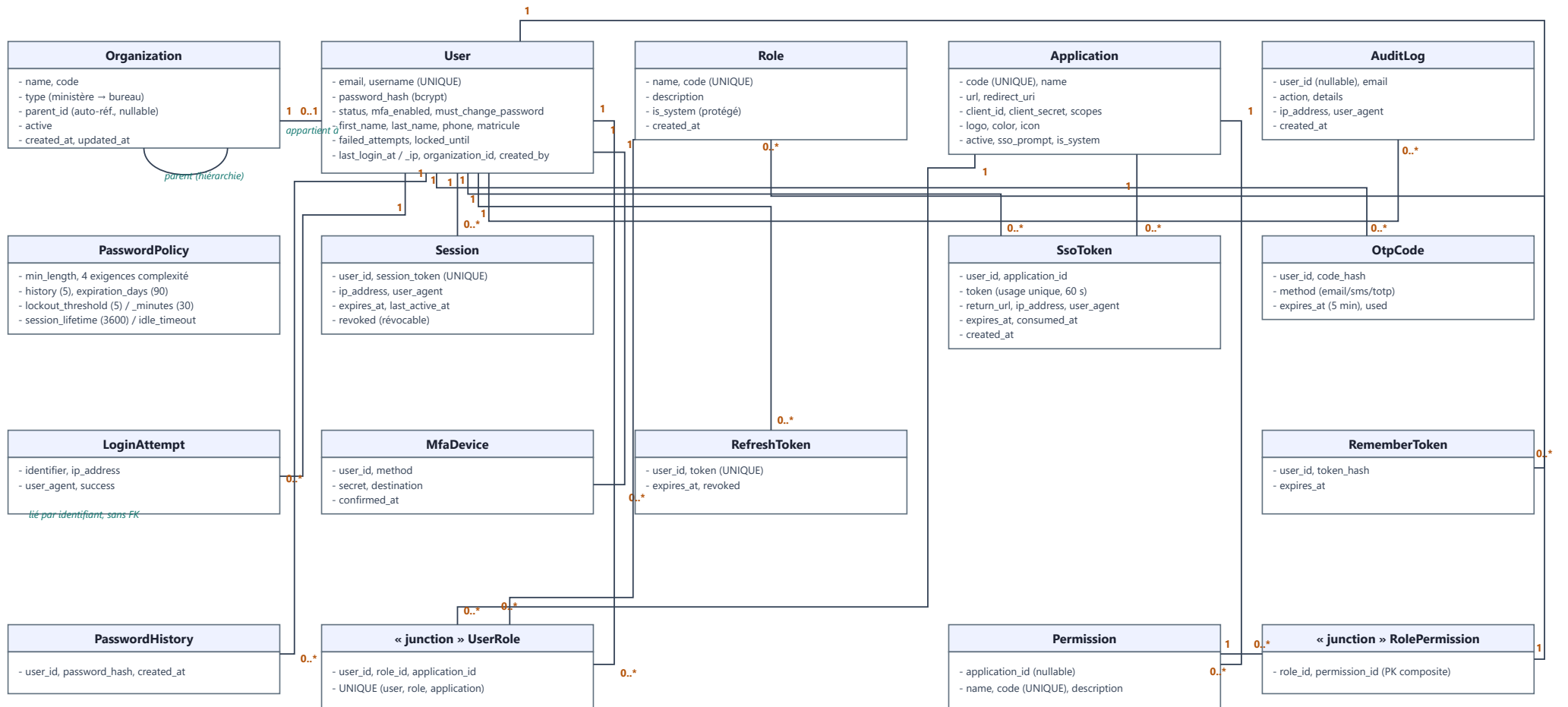
- API JSON : `api/login.php` , `api/sso/validate.php` , `api/sso/app_config.php` , `api/userinfo.php` , gestion des utilisateurs/rôles/permissions/applications/sessions, audit, réinitialisation de mot de passe, MFA (envoi/vérification).
- Bibliothèque client `client/iam_client.php` (helper `iam_client_requires_prompt()` , etc.) et `client/sso_handler.php` .
- **Serveur d'autorisation OAuth 2.0 / OIDC** (free flow Authorization Code + PKCE) : `oauth.php` (point d'autorisation + consentement), `api/oauth/token.php` (authorization_code, client_credentials, refresh_token avec rotation), `api/oauth/userinfo.php` , `api/oauth/jwks.php` , découverte `.well-known/openid-configuration.php` (racine et API). Émission

d' `id_token` signé RS256, refus journalisé (`access_denied`), PKCE par défaut.

3. Diagramme de classes

Le diagramme ci-dessous modélise le schéma relationnel `iam_local` (`sql/schema.sql`) — dont les entités du volet identité, RBAC, sessions/SSO, MFA et audit ; s'y ajoutent depuis les derniers lots : `groups` , `user_groups` , `oauth_clients` , `oauth_codes` , `oauth_keys` (22 entités au total, 6 volets).

Volet	Entités
Identité & organisation	<code>organizations</code> , <code>users</code> , <code>password_history</code>
Politique & gouvernance (RBAC)	<code>password_policy</code> , <code>roles</code> , <code>permissions</code> , <code>user_roles</code> , <code>role_permissions</code>
Annuaire & groupes	<code>`groups`</code> , <code>user_groups</code>
Sessions & SSO	<code>sessions</code> , <code>sso_tokens</code> , <code>refresh_tokens</code> , <code>remember_tokens</code>
MFA & sécurité	<code>mfa_devices</code> , <code>otp_codes</code> , <code>login_attempts</code>
OAuth 2.0 / OIDC	<code>oauth_clients</code> , <code>oauth_codes</code> , <code>oauth_keys</code>
Audit	<code>audit_logs</code>



3.1 Récapitulatif des associations

Association	Source	Cible	Cardinalité	Commentaire
Organization → User	organizations	users	1 — 0..1	organization_id nullable (ON DELETE SET NULL)
Organization → Organization	parent_id	—	0..1 — 0..*	auto-référence hiérarchique Ministère → Bureau
User → Session / RefreshToken / RememberToken	users	sessions · refresh_tokens · remember_tokens	1 — 0..*	suppression en cascade (ON DELETE CASCADE)

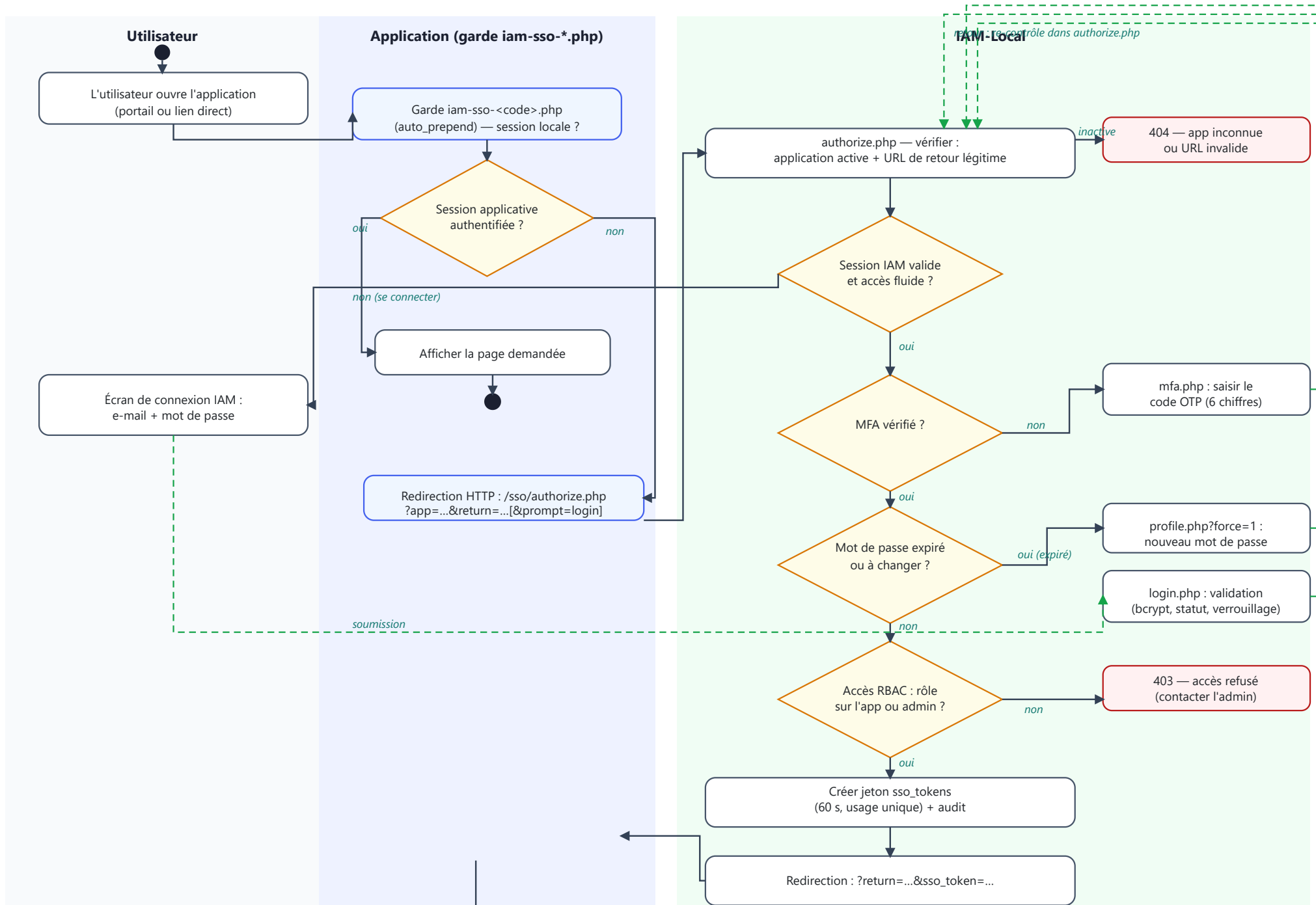
User → MfaDevice / OtpCode	users	mfa_devices · otp_codes	1 — 0..*	MFA : dispositifs enregistrés + codes OTP en attente
User → LoginAttempt	users (par identifiant)	login_attempts	1 — 0..*	anti force brute, journal de tentatives (sans FK)
User → PasswordHistory	users	password_history	1 — 0..*	historique anti-réutilisation (history = 5)
User → AuditLog	users	audit_logs	1 — 0..*	user_id nullable (ON DELETE SET NULL)
User → Application	users ⇌ applications	user_roles	0..* — 0..*	N..N à 3 clés : un rôle par utilisateur et par application
Role → Permission	roles ⇌ permissions	role_permissions	0..* — 0..*	N..N résolu par table de jonction
Application → Permission	applications	permissions	1 — 0..*	permissions par application (nullable)
Application → SsoToken	applications	sso_tokens	1 — 0..*	jeton SSO à usage unique (60 s)

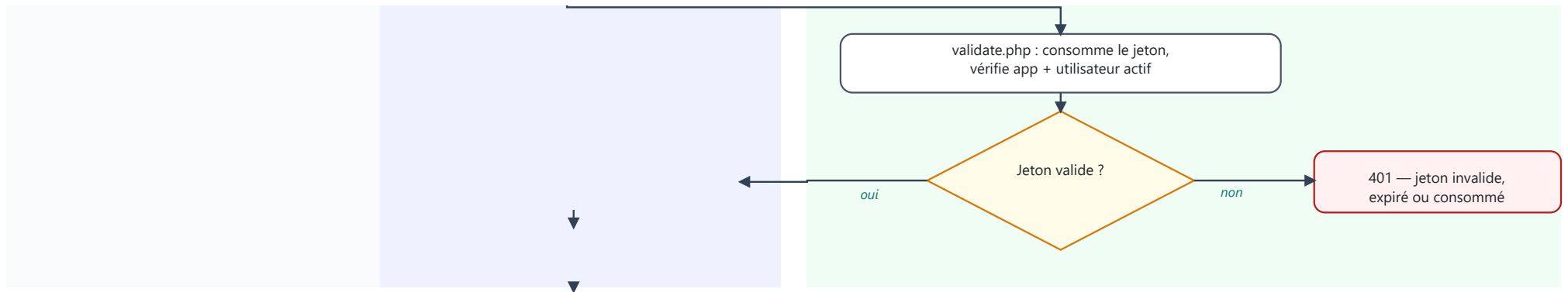
4. Diagrammes d'activité

Deux diagrammes d'activité (UML) décrivent les flux critiques : **4.1** le flux SSO de bout en bout (portail → application → IAM) et **4.2** la phase d'authentification avec MFA par e-mail et changement de mot de passe. Les pointillés représentent le retour d'une branche forcée (re-contrôle dans `authorize.php`).

4.1 Flux SSO de bout en bout

L'utilisateur accède à une application protégée par le garde `iam-sso-<code>.php` (chargé via `auto_prepend_file`). Si la session locale de l'application est absente, `sso/authorize.php` contrôle la session IAM, le MFA, le mot de passe et l'accès RBAC, puis émet un jeton à usage unique validé par `api/sso/validate.php` .

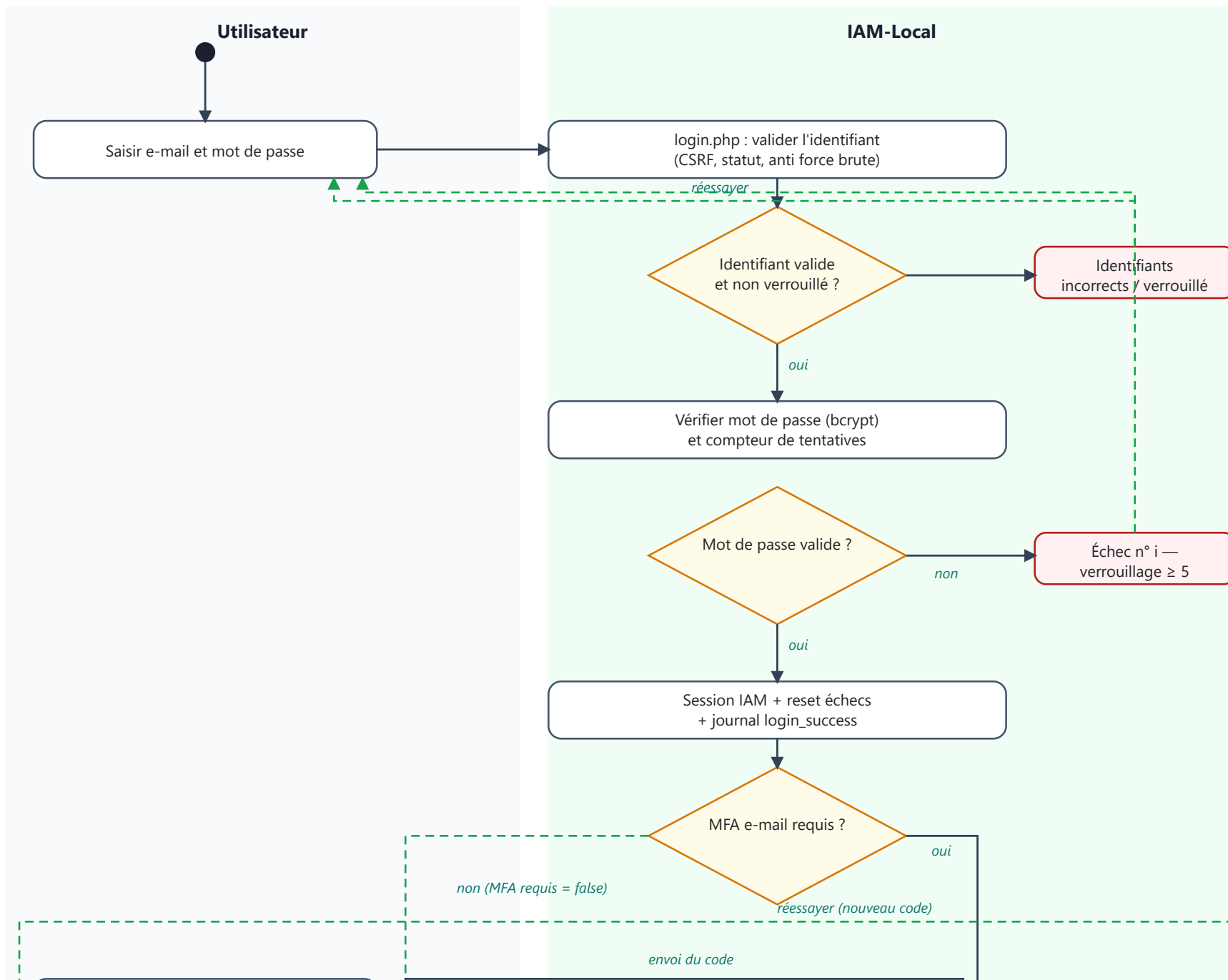


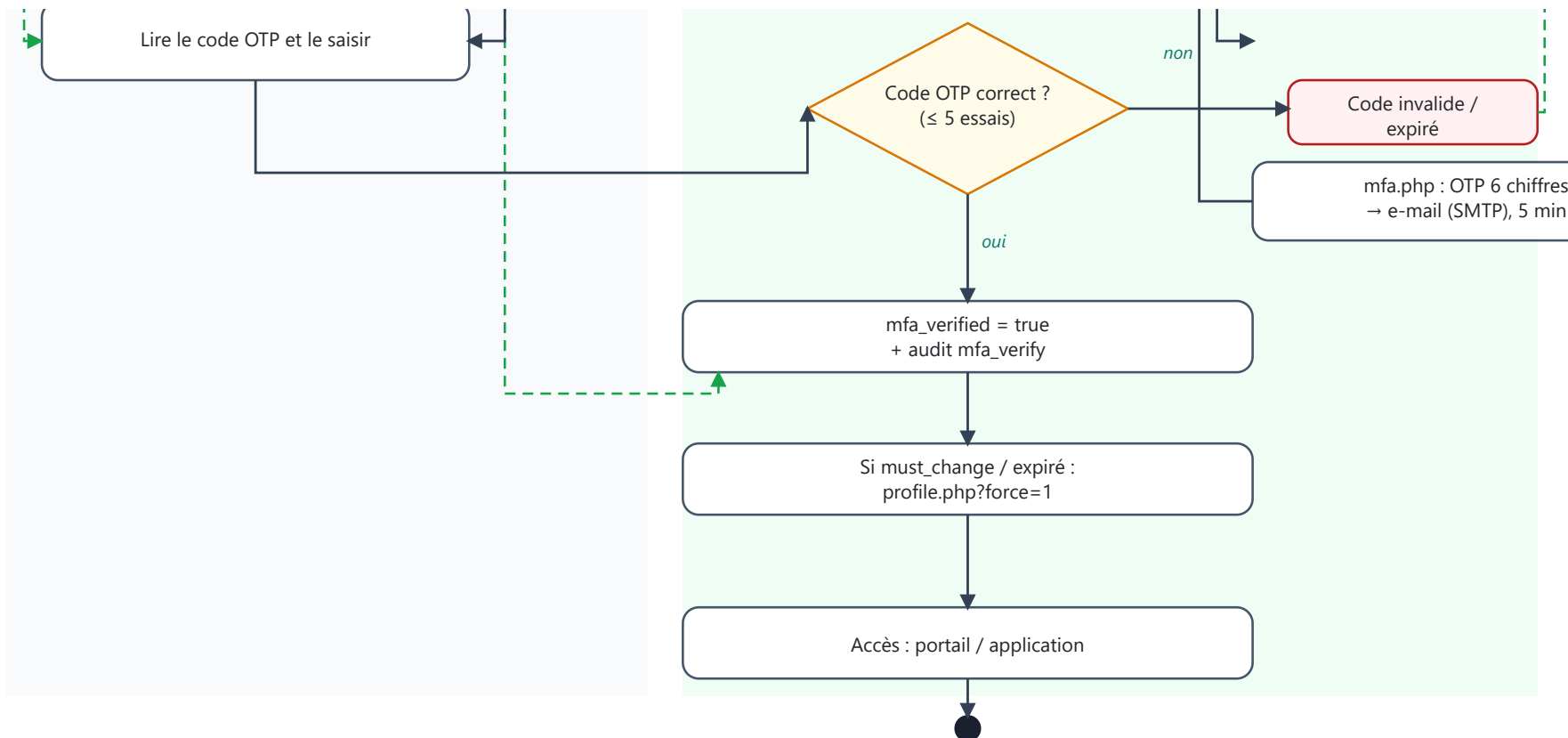


Le chemin en pointillés ramène l'utilisateur vers `authorize.php` après une action forcée (login, MFA, changement de mot de passe) : les contrôles sont renouvelés, puis un jeton `sso_tokens` est émis et validé par la garde auprès de `api/sso/validate.php` (la branche « oui » de D1 devient active pour les accès suivants).

4.2 Authentification, MFA par e-mail et changement de mot de passe

Flux interne de `login.php` puis `mfa.php` : anti force brute (5 échecs → verrouillage 30 min), code OTP à 6 chiffres (5 minutes, 5 essais, envoi SMTP), puis bascule obligatoire vers `profile.php?force=1` si `must_change_password` ou expiration atteinte.





Les boucles en pointillés relancent la saisie ; le « non » après la validation du mot de passe revient à l'écran de connexion. Un nouvel échec incrémente `login_attempts` ; dès le seuil de 5, le compte est verrouillé 30 minutes (`locked_until`). Une fois MFA vérifié et mot de passe à jour, l'accès est transmis au portail ou, via le +SSO, au retour de `authorize.php`.

5. Architecture technique et sécurité

5.1 Pile technique

Composant	Choix
Langage	PHP natif (aucun framework), PHP 8.x
Base de données	MySQL 8 (PDO), base <code>iam_local</code> , caractère utf8mb4

Sessions	Sessions PHP (cookie <code>iam_session</code>), régénération d'ID, HttpOnly, Strict Mode
Mots de passe	bcrypt (<code>password_hash</code>), historique anti-réutilisation, expiration configurable
MFA	OTP par e-mail (SMTP : Gmail/Outlook, clé applicative) ; affichage en mode démonstration si SMTP non configuré
Jeton SSO	Jeton aléatoire à usage unique (<code>sso_tokens</code>), durée de vie 60 s, consommé après usage
OAuth 2.0 / OIDC	Clé de signature RSA 2048 (RS256) auto-générée (<code>oauth_keys</code>), exposition JWKS, id_token signé, refresh tokens rotatifs
Connexion OTP	Code à usage unique sur e-mail ou téléphone (<code>otp_login.php</code>), verrouillage après échecs répétés

5.2 Mesures de sécurité

- Anti force brute positionnable (verrouillage après 5 échecs / 30 min), journalisé.
- Anti force brute **par adresse IP** sur les pages et API de connexion : 60 échecs / 15 min (paramétrable en constantes).
- Origines CORS d'accès à l'API paramétrables (`settings.cors_origins` , défaut `*`).
- Géolocalisation optionnelle des adresses IP publiques dans l'audit (service ip-api.com, sans stockage ; IP locales = « Réseau local »).
- Protection CSRF sur toutes les actions d'administration (jeton de session).
- Échappement HTML systématique (`htmlspecialchars`) sur les saisies restituées.
- Vérification de l'hôte dans l'URL de retour SSO (anti redirection ouverte).
- CURL côté application : vérification SSL activée (`CURLOPT_SSL_VERIFYPEER`).
- Journal d'audit horodaté : connexions, MFA, SSO, admin (création, suppression, droits...).
- Rôles système non supprimables, applications système non supprimables.
- OAuth 2.0 : validation stricte de la redirection (anti redirection ouverte), PKCE S256 (ou plain) comparé via `hash_equals` , codes d'autorisation à usage unique, rotation des refresh tokens (l'ancien est révoqué), scopes sur liste blanche, clé signée RS256 + rotation.

6. Environnement et exigences

Poste	Exigence
Serveur	WAMP (Apache + PHP 8 + MySQL 8), racine <code>C:\wamp64\www</code>
Base	<code>iam_local</code> — schéma fourni (<code>sql/schema.sql</code>), installateur web <code>install.php</code>
Applications intégrées (SSO)	Gestion de présence (<code>GestionPresnceAdenIAM</code>), Gestion de salaire (<code>GestionSalaireAden</code>), EduCongo2, etc. — garde déployé par l'admin
Mots de passe de démonstration	documentés dans <code>docs/guide_iam.html</code> (admin, RH présence, agent salaire)

7. Livrables et jalons

Lot	État
Référentiel utilisateurs, organisations, RBAC	Livré
Authentification + politique de mot de passe + anti force brute	Livré
MFA par e-mail (SMTP + mode démo + test)	Livré
SSO à jeton unique + garde auto-déployable + portail	Livré
Paramètre <code>sso_prompt</code> (ressaisie sur lien direct)	Livré
Console d'administration graphique complète	Livré
Attributs de poste (direction/service/fonction) + import CSV + annuaires de groupes	Livré
Connexion sans mot de passe par e-mail ou téléphone (OTP)	Livré
Serveur d'autorisation OAuth 2.0 / OIDC (PKCE, JWKS, id_token, refresh rotatifs, discovery)	Livré
Guide d'utilisation (PDF + HTML), présent cahier des charges	Livré